

Partial Differential Equations 3 (Numerical)

Lecture 3 - Academic Year 2025/2026

Prof Halim Kusumaatmaja
Email: halim.kusumaatmaja@ed.ac.uk
Semester 2, Edinburgh Engineering

Aims and Objectives

1. To describe Successive Over-Relaxation (SOR).
2. To discuss convergence of a numerical scheme.
3. To discuss comparisons between Jacobi, Gauss-Sidel and SOR.

Successive Over-Relaxation (SOR)

Both in Jacobi and Gauss-Sidel iterations in Lecture 2, you may have noticed that our algorithms to update $u_{i,j}^{m+1}$ do not exploit our previous iteration value $u_{i,j}^m$ in any way. This is the information that we will exploit in SOR that makes SOR more effective. The algorithm for SOR can be written as:

$$u_{i,j}^{m+1} = (1 - \omega) u_{i,j}^m + \omega u_{i,j|GS}^{m+1} \quad (1)$$

$$\text{or } u_{i,j}^{m+1} = u_{i,j}^m + \omega (u_{i,j|GS}^{m+1} - u_{i,j}^m) \quad (2)$$

$$\text{with } u_{i,j|GS}^{m+1} = \frac{1}{2(1 + \beta^2)} (u_{i+1,j}^m + u_{i-1,j}^{m+1} + \beta^2 (u_{i,j+1}^m + u_{i,j-1}^{m+1})).$$

Eqs. (1) and (2) are equivalent and they are common ways of writing the SOR scheme. Eq. (2) is useful to consider. Logically we can consider the term $(u_{i,j|GS}^{m+1} - u_{i,j}^m)$ as the correction at a given iteration to the current value. The key idea behind SOR is that we over-relax: since $(u_{i,j|GS}^{m+1} - u_{i,j}^m)$ is the direction of travel to get to the solution, we step in this direction as much as we can. Fig. 1 illustrates this idea pictorially, comparing Gauss-Sidel and SOR algorithms.

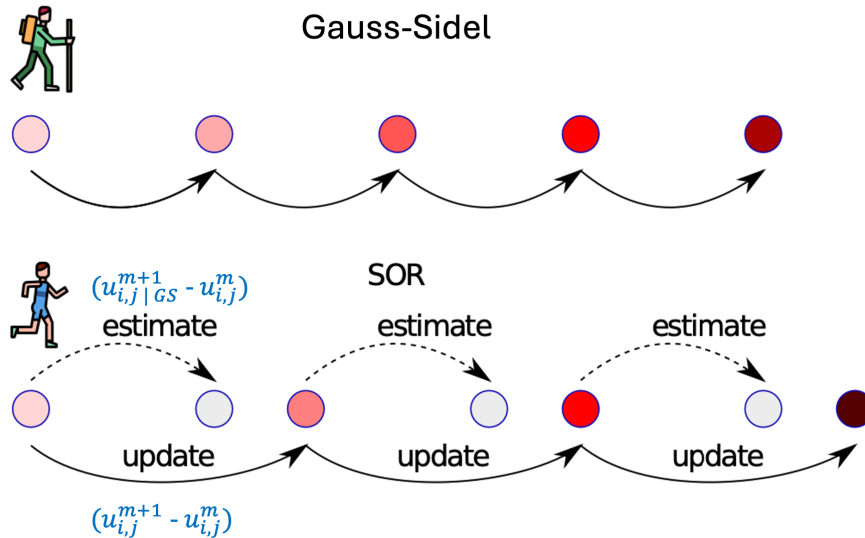


Figure 1: Illustrative comparison of Gauss-Sidel vs SOR schemes.

Mathematically, we can consider the following choices of ω :

- If $\omega = 1$, we simply go back to the Gauss-Sidel iteration.
- If $0 < \omega < 1$, we have a **successive under-relaxation** scheme.
- If $1 < \omega < 2$, we have a **successive over-relaxation** scheme. The optimal value for ω is normally problem dependant and has to be found by trial-and-error. However, on a rectangular $n_x \times n_y$ grid, we can use

$$\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - \lambda}}; \quad \text{with } \lambda = \frac{1}{4} \left[\cos\left(\frac{\pi}{n_x}\right) + \cos\left(\frac{\pi}{n_y}\right) \right]^2. \quad (3)$$

- If $\omega > 2$, the scheme is unstable, since we have overdone our over-relaxation.

We can now write the pseudocode for SOR as follows:

```

set  $m = 0$ 
initialise  $u_{ij}^{m=0}$  for all  $i, j$ ; this is your best or simplest guess usually
compute  $\beta = \Delta x / \Delta y$ 
compute  $\omega_{\text{opt}}$  via Eq. (3)
repeat
    apply boundary conditions
    compute Eq. (1) for all  $i, j$ 
    increment  $m$ 
until  $\max |u_{ij}^m - u_{ij}^{m-1}| < \epsilon$ ; or any other stop tolerance criterion

```

You will implement SOR in computing workshop 2, and hence it is important that you understand the algorithm.

Exercise 1: In the web interactive tool (see link below), you can compare SOR computations when using the optimal Ω , as well as lower and higher values of Ω . Comment on the performance of the three cases. In this problem, we set $u = 1$ on the boundary nodes and initialise $u = 0$ everywhere else, just like the example “Effect of Number of Iterations” on the Jacobi iteration interactive tool.

Scheme Convergence

In this section we discuss how we decide that the grid is fine enough for a **Mesh independent solution** to be obtained. One way is to compare with the analytical result, but in general we do not have the analytical solution - if we do, we do not need the numerical solution!

When comparing solutions for different grids, it is wise to choose a sequence of grids that allows for easy comparison. This is illustrated in Fig. 2 where we ‘double the resolution’ at every grid level. Note that the same mesh points appear on every grid (shown in green) so that we can easily compare local information on the grid points if desired. Mathematically, for the 1D grid example here, the number of grid points is given by $n_x = 2^\alpha \beta + 1$ where for the coarsest grid ($\alpha = 1$), the number of grid points is $n_x = 2\beta + 1$, and we have more points for finer grid (higher α). Since we usually employ square or cubic grids in 2D and 3D, we can extend the logic easily to these higher dimensions.

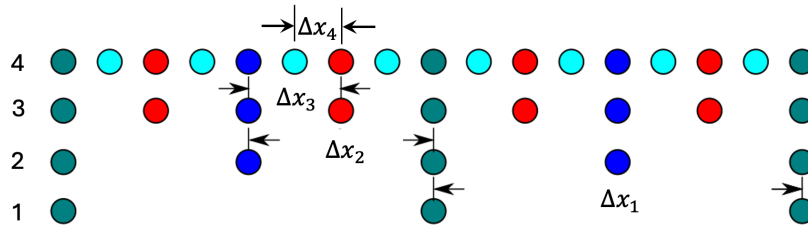


Figure 2: An illustration of a suitable sequence of grid.

We also have an option whether to compare local variables on grid points or integrated quantities. The former can be better if we know there are specific regions in the computational domain that are particularly important or sensitive. Otherwise, the latter is recommended as

it better captures the entirety of the computational domain. In real engineering problems, examples of suitable integrated quantities (depending on the problem of course) include total lift force, or the coefficient of lift, over an airfoil; total drag force, or the coefficient of drag, on a car; maximum deflection of a cantilevered beam; or discharge rate out of a pipe. In our example problem, we can choose a simple measure integrated over the grid domain Ω :

$$U = \int_{\Omega} u^2 dx dy = \sum_{i,j} u_{i,j}^2 \Delta x \Delta y. \quad (4)$$

Once we have the results for different grid resolutions, there are a number of analysis we can harness to know the quality of our results. Here we will look into three simple but powerful approaches.

First, we can look at the apparent order of convergence p . We can define the error E as the difference between the discrete solution and the exact solution:

$$\begin{aligned} E &= U(\Delta x) - U_{\text{exact}} = C(\Delta x)^p + \text{higher order terms,} \quad \text{or} \\ \log E &= p \log \Delta x + \log(C). \end{aligned} \quad (5)$$

Here, the important information to extract is p . Note that we do not actually need to know what U_{exact} is so long as we have three or more data points.

Second, we can employ Richardson extrapolation to obtain a higher-order estimate of the continuum value (the value of the integral quantity on a mesh with zero grid spacing), $U(\Delta x \rightarrow 0)$, from a series of lower-order discrete values. We may fit the data to a polynomial

$$U(\Delta x) = U(\Delta x \rightarrow 0) + g_1 \Delta x + g_2 \Delta x^2 + g_3 \Delta x^3 + \dots \quad (6)$$

Third, we can take advantage of the grid convergence index, defined as

$$GCI_{12} = \frac{F_s |\epsilon_{12}|}{r^p - 1} \quad (7)$$

between two meshes with resolutions Δx_1 and Δx_2 (the convention for this equation is that $\Delta x_1 < \Delta x_2$). F_s is a factor of safety. The factor of safety is recommended to be $F_s = 3.0$ for comparisons of two grids and $F_s = 1.25$ for comparisons over three or more grids (usually the case). The relative error between two grids is $\epsilon_{12} = (U(\Delta x_2) - U(\Delta x_1))/U(\Delta x_1)$, r is the ratio of mesh size (typically, $r = \Delta x_2/\Delta x_1$), and p is the apparent order of convergence defined previously. We say the solution is in the asymptotic range and that we have a grid independent solution if

$$\frac{GCI_{23}}{r^p GCI_{12}} \approx 1. \quad (8)$$

Exercise 2: Assuming the numerical results for different grid resolutions follow the scaling in Eq. (5), show that

$$\frac{GCI_{23}}{r^p GCI_{12}} = \frac{U(\Delta x_1)}{U(\Delta x_2)} \approx 1. \quad (9)$$

Comparison between Jacobi, Gauss-Sidel and SOR

We now want to perform the performance between Jacobi iteration, Gauss-Sidel and SOR. This is done for the example problem previously described in Lecture 2, where we solve the 2D Laplace equation on a grid defined by $0 \leq x \leq 2$ and $0 \leq y \leq 1$ subject to the following boundary condition:

$$\begin{aligned} u(x, 0) &= 0 & \text{for } 0 \leq x \leq 2 \\ u(x, 1) &= 0 & \text{for } 0 \leq x \leq 2 \\ u(0, y) &= 0 & \text{for } 0 \leq y \leq 1 \\ u(2, y) &= \sin(2\pi y) & \text{for } 0 \leq y \leq 1 \end{aligned}$$

For convenience, we have tabulated the integrated quantity U as a function of the grid size, which you can find below for convenience together with the run time. Based on these results, we can find the apparent order of convergence, the Richardson extrapolation value, and when we have a grid independent solution.

Table 1: Results comparison between Jacobi, Gauss-Sidel and SOR schemes. For completeness, here as a convergence criterion, we have used $|U^{m+1} - U^m| < 10^{-10}$. T is the time needed to get the desired convergence in seconds, while I is the number of iterations required. Note that these provide different information because the cost/time to run 1 iteration is not the same for Jacobi, Gauss-Sidel and SOR schemes.

No.	Grid Points	Δx	U	$T_{J(\text{array})}(\text{s})$	$T_{J(\text{loop})}(\text{s})$	$T_{GS}(\text{s})$	$T_{SOR}(\text{s})$
1	1281 X 641	0.00156	0.0401	486	-	43200	831
2	641 X 321	0.00313	0.0405	25.9	4510	3780	93.3
3	321 X 161	0.00625	0.0413	1.16	227	158	10.6
4	161 X 81	0.0125	0.0429	0.0925	17.2	9.72	1.22
5	81 X 41	0.0250	0.0464	9.20×10^{-3}	1.18	0.607	0.102
6	41 X 21	0.0500	0.0538	1.29×10^{-3}	7.62×10^{-2}	3.71×10^{-2}	1.20×10^{-2}
7	21 X 11	0.100	0.0710	3.04×10^{-4}	3.85×10^{-3}	2.33×10^{-3}	1.40×10^{-3}
8	11 X 6	0.200	0.112	9.18×10^{-5}	2.46×10^{-4}	1.74×10^{-4}	1.15×10^{-4}

No.	Grid Points	I_J	I_{GS}	I_{SOR}
1	1281 X 641	157480	89799	1451
2	641 X 321	39370	22469	657
3	321 X 161	9842	5626	306
4	161 X 81	2460	1411	143
5	81 X 41	615	355	49
6	41 X 21	154	89	24
7	21 X 11	38	23	12
8	11 X 6	9	6	2

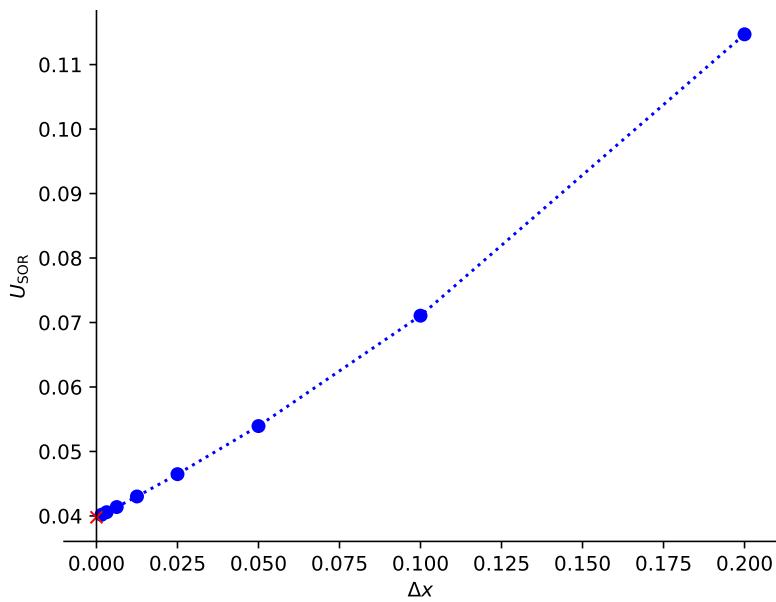


Figure 3: Variation of U_{SOR} with Δx for SOR.

Fig. 3 shows how U_{SOR} varies with Δx . Using Eq. (6) we can extrapolate to $\Delta x = 0$ to find $U(\Delta x \rightarrow 0) = 0.0398$. Fitting the last 4 data points with Eq. (5), we find $p = 1.00$ (we do this

because we expect the larger Δx to be inaccurate). The order of convergence is low, roughly first order. Moreover, we can carry out the grid convergence index analysis and the results are tabulated below.

Grid i	$GCI_{i,i+1}$	$GCI_{i+1,i+2}/(r^p GCI_{i,i+1})$
1	0.0125	0.990
2	0.0247	0.981
3	0.0484	1.053
4	0.1020	0.977
5	0.1994	1.002
6	0.3996	0.903
7	0.7218	N/A

To have results which are mesh independent/in the asymptotic range, we want $GCI_{i+1,i+2}/(r^p GCI_{i,i+1}) \approx 1$. Clearly grids 6, 7, and 8 are too coarse represented by $GCI_{i+1,i+2}/(r^p GCI_{i,i+1}) = 0.903$. After this, it is a judgement call in terms of how tight we want to enforce our criterion.

Exercise 3: What can you say about the algorithm (Jacobi vs Gauss-Sidel vs SOR) that performs best based on these analyses?

Additional Materials

- Interactive tool: <https://kusumaatmaja.github.io/PDE3/examples>

Version Control:

- version 1.0 on 20 December 2025